Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Модель данных

Сущности, инварианты, справочники и миграции · версия 0.6.0

29 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Обзор 4

2.1 Организация 5

2.2 Отпуска и режим работы 6

2.3 Служебные данные 6

2.4 Кадровый учёт 7

3 Словарь данных 8

4 Инварианты 18

5 Индексы 21

6 Служебные таблицы масштабирования 22

7 Производственный календарь 23

8 Миграция данных прототипа 24

9 Изменение схемы 25

1 О документе

Документ описывает модель данных «Планировщика отпусков»: сущности и связи, словарь данных, инварианты и способ их обеспечения, производственный календарь, миграцию данных прототипа и правила изменения схемы.

Единственный источник DDL — `server/prisma/schema.prisma`; SQL-миграции — `server/prisma/migrations/`. Словарь данных ниже сформирован из схемы автоматически (`tools/docs/gen_tables.py`).

2 Обзор

Данные хранятся в PostgreSQL 17 в 24 таблицах четырёх групп:

ГРУППА	ТАБЛИЦЫ	НАЗНАЧЕНИЕ
Организация	`employees`, `departments`, `projects`, `groups`, `group_members`	Кто работает, в каком отделе и проекте, кто чей руководитель
Отпуска и режим	`vacation_requests`, `request_events`, `notifications`, `work_patterns`, `work_overrides`	Заявки с историей, уведомления, офис и удалёнка
Кадровый учёт	`approval_delegations`, `vacation_notices`, `planning_campaigns`, `schedule_approvals`	Замещение руководителя, извещения о начале отпуска, кампания планирования и утверждённый график (с v0.6.0)
Служебные	`audit_events`, `refresh_tokens`, `login_state`, `login_events`, `org_settings`, `calendars`, `email_outbox`, `job_runs`, `data_version`, `login_attempts`	Журнал изменений, сеансы, состояние и журнал входов, правила организации, праздники, очередь писем, плановые задачи, версия данных графика, окна лимита входа

2.1 Организация

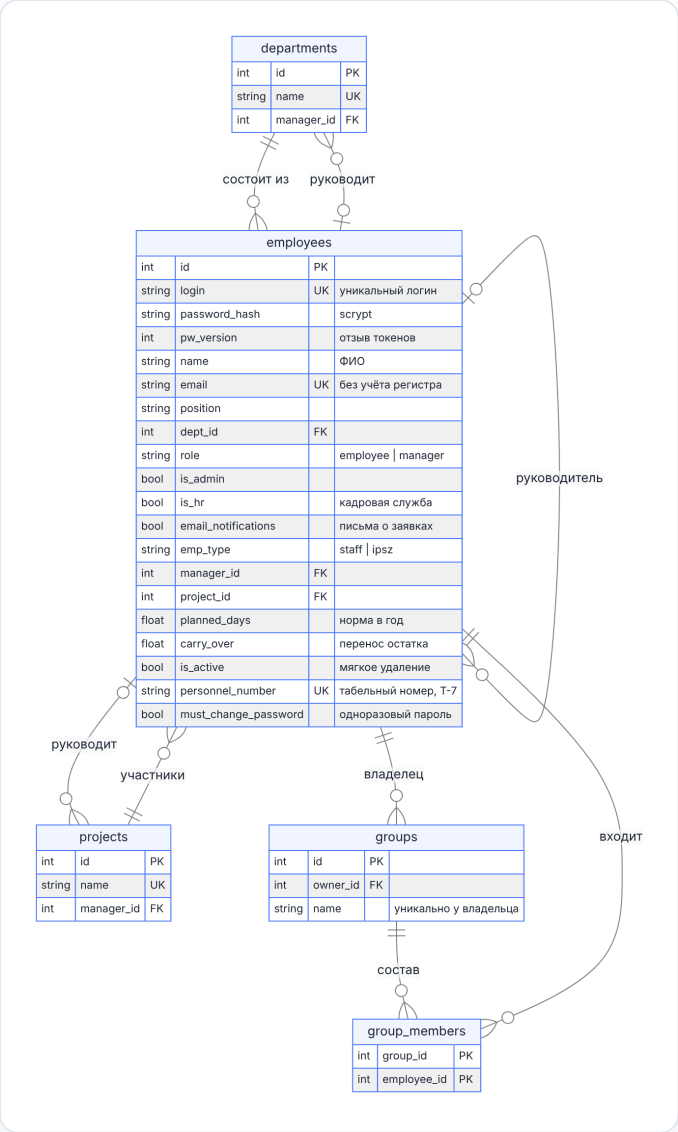


Рис. 1. ER: сотрудники, отделы, проекты, группы

Руководитель сотрудника (`employees.manager_id`) согласует его заявки и управляет его режимом работы. Признак `is_hr` даёт права кадровой службы, `email` и `email_notifications` — адрес и согласие на письма. Руководитель отдела (`departments.manager_id`) выгружает отпуска отдела. Сотрудник не удаляется физически: увольнение снимает признак `is_active` , история отпусков и журнал сохраняются. Табельный номер `personnel_number` (графа 4 формы Т-7) уникален во всей организации, включая уволенных. `must_change_password` — пароль выдан кадровой службой и должен быть сменён при первом входе.

2.2 Отпуска и режим работы

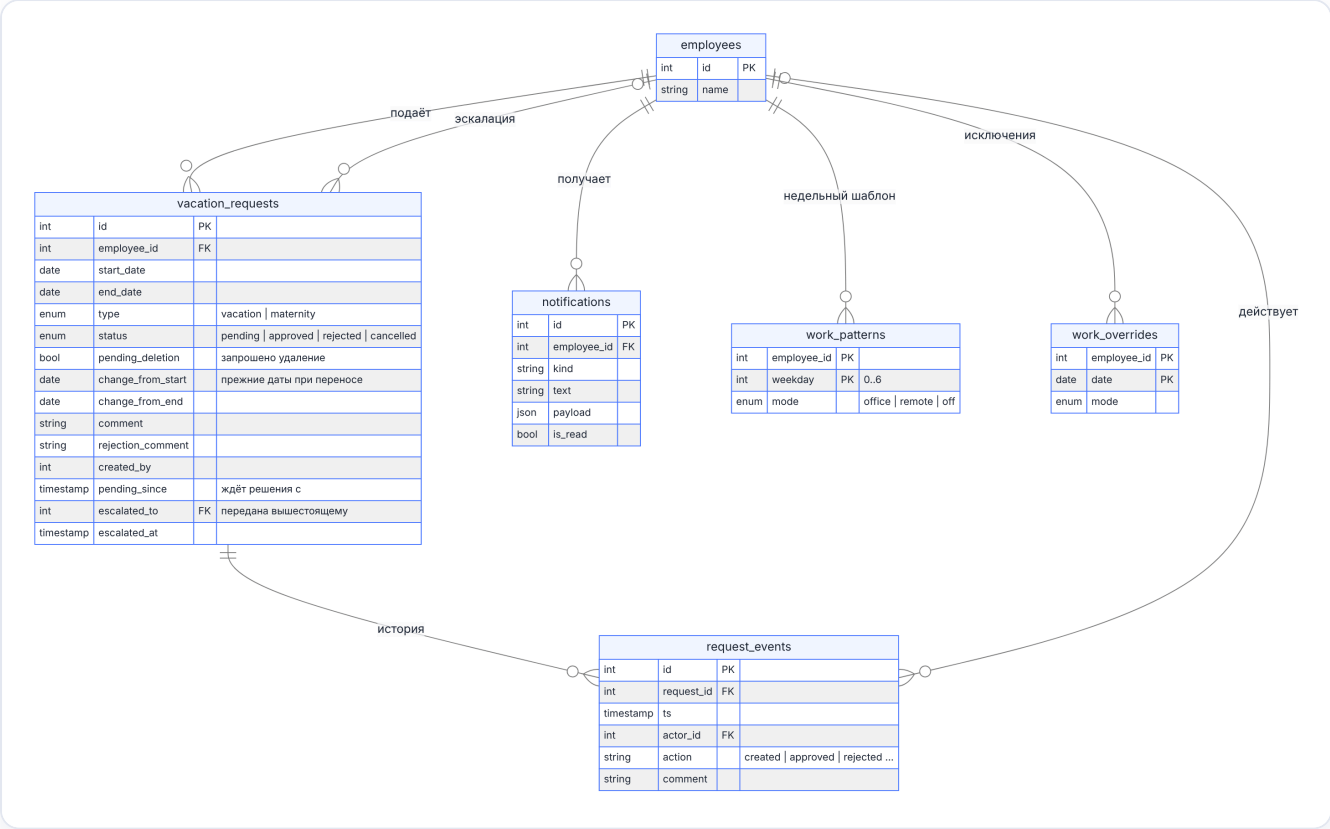


Рис. 2. ER: заявки, история, уведомления, режим работы

Заявка хранит актуальные даты; при переносе согласованного отпуска прежние даты лежат в `change_from_start/end` до решения руководителя. Запрос удаления — флаг `pending_deletion`. История действий по заявке — `request_events` (только дополняется). `pending_since` — с какого момента заявка ждёт решения (создание, перенос, запрос удаления); `escalated_to` и `escalated_at` — кому и когда она передана эскалацией. Режим работы: недельный шаблон `work_patterns` (день недели → офис/удалёнка/не работает) и разовые исключения `work_overrides` на даты.

2.3 Служебные данные

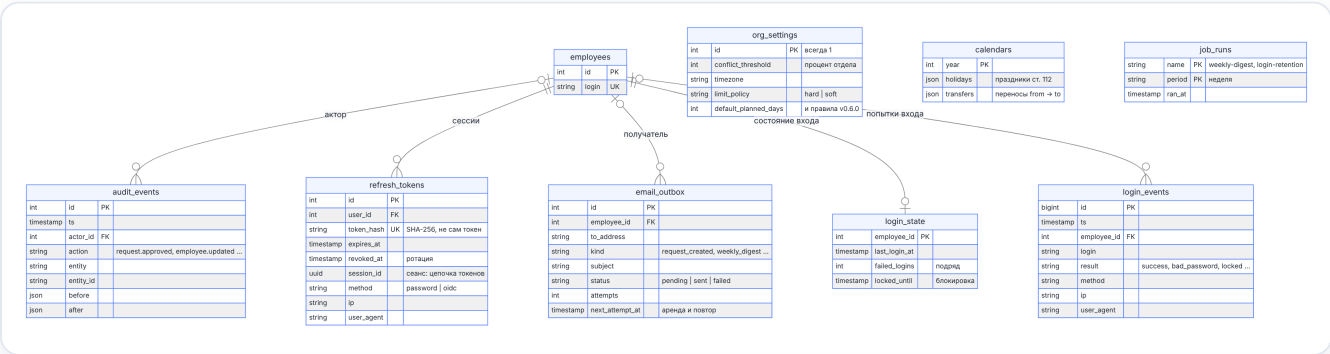


Рис. 3. ER: аудит, сессии, настройки, календарь

Сеанс — цепочка строк `refresh_tokens` с общим `session_id`, способом входа, адресом и браузером. Состояние входа (последний вход, неудачные попытки подряд, блокировка) — в `login_state`, отдельно от `employees`: запись в `employees` меняет версию данных графика и сбрасывала бы кэш графика при каждом входе. Журнал входов `login_events` хранится `login_retention_days` дней. Реквизиты организации и подписанты формы Т-7 — в `org_settings`.

2.4 Кадровый учёт

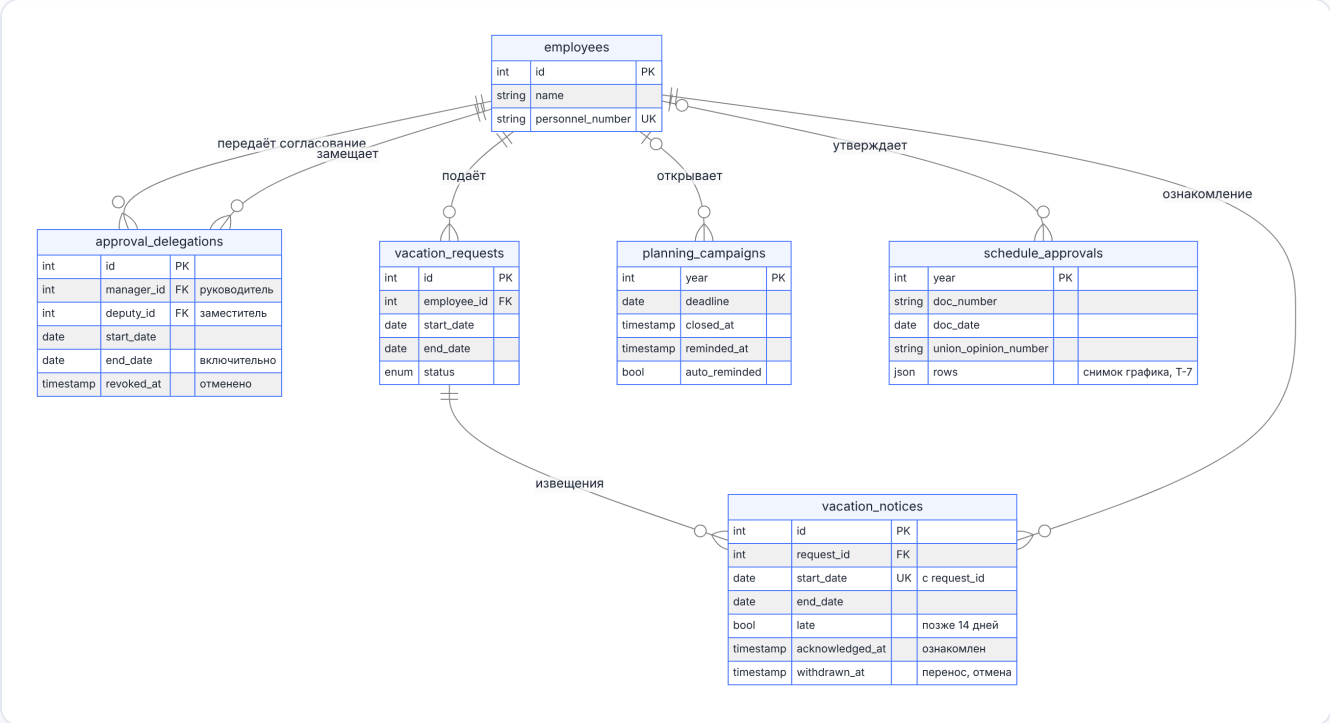


Рис. 4. ER: замещение, извещения, кампания и утверждённый график

Замещение (`approval_delegations`). Руководитель (или кадровая служба за него) передаёт права согласующего заместителю на период включительно; отменённое замещение (`revoked_at`) остаётся в истории.

Извещения (`vacation_notices`). Строка на согласованный отпуск и дату его начала: когда извещён, позже ли двух недель (`late`), когда сотрудник подтвердил ознакомление, когда извещение отозвано (перенос, отмена, увольнение). Перенос на другие даты — новая строка; прежняя остаётся в журнале как история.

Кампания планирования (`planning_campaigns`). Одна строка на год: срок, кто и когда открыл и закрыл, последнее напоминание, отметка автоматического напоминания перед сроком.

Утверждённый график (`schedule_approvals`). Одна строка на год: номер и дата документа, мнение профсоюза, кто и когда утвердил и снимок строк графика в JSONB — сотрудник, должность, отдел, табельный номер, даты и дни каждого согласованного отпуска на момент утверждения (ADR-016). Снимок не меняется; переносы и отмены после утверждения форма Т-7 берёт из текущих заявок.

3 Словарь данных

`employees` — Сотрудники: учётные записи, роли, нормы дней

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`login`	String	UK	
`password_hash`	String		
`pw_version`	Int		
`name`	String		
`email`	String	null	уникальность без учёта регистра — частичным индексом в миграции employee_email
`position`	String	null	
`dept_id`	Int	null	
`role`	String		employee | manager
`is_admin`	Boolean		
`is_hr`	Boolean		кадровая служба: сотрудники, календарь, импорт, выгрузки, аудит
`emp_type`	String		staff | ipsz
`manager_id`	Int	null	
`projectId`	Int	null	
`planned_days`	Float		
`carry_over`	Float		
`is_active`	Boolean		
`email_notifications`	Boolean		отписка — в профиле

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`must_change_password`	Boolean		пароль выдан при создании или сбросе (v0.6.0)
`password_changed_at`	DateTime	null	
`personnel_number`	String	UK null	табельный номер — графа 4 формы Т-7 (v0.6.0)
`created_at`	DateTime		
`updated_at`	DateTime		

`departments` — Отделы и их руководители

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`name`	String	UK	
`manager_id`	Int	null	

`projects` — Проекты и руководители проектов

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`name`	String	UK	
`manager_id`	Int	null	

`groups` — Личные группы для фильтров графика

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`owner_id`	Int		
`name`	String		

`group_members` — Состав личных групп

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`group_id`	Int		
`employee_id`	Int		

`vacation_requests` — Заявки на отпуск и их состояние

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`employee_id`	Int		
`start_date`	DateTime		
`end_date`	DateTime		
`type`	RequestType		
`status`	RequestStatus		
`pending_deletion`	Boolean		
`change_from_start`	DateTime	null	
`change_from_end`	DateTime	null	
`comment`	String	null	
`rejection_comment`	String	null	
`created_by`	Int		
`created_at`	DateTime		
`updated_at`	DateTime		
`pending_since`	DateTime	null	
`escalated_to`	Int	null	
`escalated_at`	DateTime	null	

`request_events` — История действий по заявке

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`request_id`	Int		
`ts`	DateTime		
`actor_id`	Int	null	
`action`	String		
`comment`	String	null	

`work_patterns` — Недельный шаблон офис/удалёнка

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`employee_id`	Int		
`weekday`	Int		0 (вс) … 6 (сб)
`mode`	WorkMode		

`work_overrides` — Разовые исключения режима работы

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`employee_id`	Int		
`date`	DateTime		
`mode`	WorkMode		

`org_settings` — Правила организации (одна строка)

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`conflict_threshold`	Int		
`timezone`	String		
`limit_policy`	String		hard | soft
`default_planned_days`	Int		
`password_min_length`	Int		

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`lockout_threshold`	Int		неудачных попыток подряд; 0 — без блокировки
`lockout_minutes`	Int		
`login_retention_days`	Int		
`escalation_days`	Int		
`notice_days`	Int		
`org_name`	String	null	
`org_okpo`	String	null	
`head_position`	String	null	
`head_name`	String	null	
`hr_head_position`	String	null	
`hr_head_name`	String	null	

`calendars` — Производственный календарь по годам

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`year`	Int	PK	
`holidays`	Json		[“2026-01-01”, ...]
`transfers`	Json		[{“from”:“2026-01-03”,“to”:“2026-01-09”}, ...]

`notifications` — Уведомления пользователей

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`employee_id`	Int		
`kind`	String		
`text`	String		
`payload`	Json	null	

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`is_read`	Boolean		
`created_at`	DateTime		

`audit_events` — Неизменяемый журнал изменений

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`ts`	DateTime		
`actor_id`	Int	null	
`action`	String		
`entity`	String		
`entity_id`	String		
`before`	Json	null	
`after`	Json	null	

`refresh_tokens` — Хеши refresh-токенов (ротация, отзыв); цепочка одного входа — сеанс

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`user_id`	Int		
`token_hash`	String	UK	
`expires_at`	DateTime		
`revoked_at`	DateTime	null	
`created_at`	DateTime		
`session_id`	String		
`session_started_at`	DateTime		
`method`	String		password | oidc
`ip`	String	null	

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`user_agent`	String	null	

`login_state` — Состояние входа сотрудника: последний вход, неудачные попытки подряд, блокировка

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`employee_id`	Int	PK	
`last_login_at`	DateTime	null	
`failed_logins`	Int		
`locked_until`	DateTime	null	

`login_events` — Журнал входов: успешные и неуспешные попытки, блокировки; срок хранения — правило организации

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`ts`	DateTime		
`employee_id`	Int	null	
`login`	String		
`result`	String		
`method`	String		
`ip`	String	null	
`user_agent`	String	null	

`email_outbox` — Очередь писем (outbox): пишется в транзакции события, отправляется фоном с повторами

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`id`	Int	PK	
`employee_id`	Int	null	
`to_address`	String		
`kind`	String		

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`subject`	String		
`body`	String		
`status`	String		pending | sent | failed
`attempts`	Int		
`next_attempt_at`	DateTime		
`last_error`	String	null	
`created_at`	DateTime		
`sent_at`	DateTime	null	

`job_runs` — Отметки плановых задач: строка (задача, период) — задача за период уже выполнена

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`name`	String		
`period`	String		
`ran_at`	DateTime		

`data_version` — Версия данных графика: 16 строк-счётчиков, увеличиваются триггерами; версия — сумма

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`shard`	Int	PK	

`login_attempts` — Окна лимита входа по минутам (нежурналируемая таблица, общая для всех процессов)

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`key`	String		
`window_start`	DateTime		
`hits`	Int		

approval_delegations — Замещение руководителя: заместитель согласует заявки его команды на период

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
id	Int	PK	
manager_id	Int		
deputy_id	Int		
start_date	DateTime		
end_date	DateTime		
created_by	Int	null	
created_at	DateTime		
revoked_at	DateTime	null	

vacation_notices — Извещения о начале отпуска под подпись (ст. 123 ТК РФ): отправка, ознакомление, отзыв

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
id	Int	PK	
request_id	Int		
employee_id	Int		
start_date	DateTime		
end_date	DateTime		
days	Float		дни отпуска без праздников (ст. 120)
sent_at	DateTime		
late	Boolean		позже чем за 14 дней: отпуск согласован поздно
acknowledged_at	DateTime	null	
withdrawn_at	DateTime	null	отпуск отменён или перенесён

planning_campaigns — Кампания планирования отпусков на год: срок, открытие и закрытие, напоминания

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`year`	Int	PK	
`deadline`	DateTime		
`opened_at`	DateTime		
`opened_by`	Int	null	
`closed_at`	DateTime	null	
`closed_by`	Int	null	
`reminded_at`	DateTime	null	
`auto_reminded`	Boolean		

`schedule_approvals` — Утверждённый график отпусков: реквизиты документа, мнение профсоюза, снимок строк для формы Т-7

ПОЛЕ	ТИП	ПРИЗНАКИ	ПРИМЕЧАНИЕ
`year`	Int	PK	
`doc_number`	String		
`doc_date`	DateTime		
`union_opinion_date`	DateTime	null	
`union_opinion_number`	String	null	
`approved_at`	DateTime		
`approved_by`	Int	null	
`rows`	Json		

4 Инварианты

ИНВАРИАНТ	ГДЕ ОБЕСПЕЧИВАЕТСЯ	ОТВЕТ ПРИ НАРУШЕНИИ
`start_date ≤ end_date`, даты в формате `YYYY-MM-DD`, не в прошлом (кроме администратора)	Сервис заявок	`400 validation/bad-date-range`
Активные (`pending`, `approved`) заявки одного сотрудника не пересекаются	Сервис заявок под блокировкой строки сотрудника (`SELECT … FOR UPDATE`): одновременные запросы проверяются по очереди	`409 overlapping-request`
При жёсткой политике лимита заявка не превышает «доступно» = норма − согласованные − на согласовании дни (без праздников)	Сервис заявок под той же блокировкой	`409 limit-exceeded`
Решение по заявке применяется один раз	Блокировка строки заявки и повторное чтение в транзакции	`409 stale-state`
У отклонённого или согласованного переноса нет `change_from_*`	Сервис заявок	—
Статусы, типы и режимы — только из перечислений	Типы `enum` PostgreSQL	ошибка БД
Уникальность логина, названия отдела и проекта, группы у владельца	Уникальные индексы	`400 validation/invalid-field`
Email сотрудника уникален без учёта регистра	Частичный уникальный индекс `lower(email)`; проверка в сервисе	`400 validation/invalid-field`; гонка — `409 stale-state`
Письмо создаётся только вместе с событием	`email_outbox` пишется в транзакции уведомления (outbox)	—

ИНВАРИАНТ	ГДЕ ОБЕСПЕЧИВАЕТСЯ	ОТВЕТ ПРИ НАРУШЕНИИ
Письмо отправляется один раз	Разбор очереди с `FOR UPDATE SKIP LOCKED` и арендой; сводка — строка `job_runs (name, period)`	—
Кадровая служба не выдаёт себе и другим служебные роли	Сервис сотрудников и план импорта	`403 auth/forbidden`
В системе остаётся хотя бы один администратор	Сервис сотрудников	`409 last-admin`
Отдел с сотрудниками не удаляется	Контроллер справочников	`409 department-not-empty`
Изменение данных и запись журнала неразделимы	Одна транзакция Prisma; в API нет изменения и удаления журнала	—
Табельный номер уникален	Уникальный индекс `personnel_number`; проверка в сервисе и плане импорта	`400 validation/invalid-field`
Refresh-токен обменивается один раз	`UPDATE … SET revoked_at … WHERE revoked_at IS NULL RETURNING` в транзакции обновления; повтор в течение 30 секунд — пара того же сеанса, позже — завершение сеанса	`401`
Счётчик неудачных попыток не теряет попытки при параллельных входах	Атомарный `INSERT … ON CONFLICT DO UPDATE` в `login_state`	—
Замещения руководителя не пересекаются по датам	Сервис под блокировкой строки руководителя (`FOR NO KEY UPDATE`)	`400 validation/invalid-field`
Заявка передаётся эскалацией один раз	`UPDATE … WHERE escalated_at IS NULL` перед уведомлением	—
Одно извещение на заявку и дату начала, в том числе из разных процессов	Уникальный индекс `(request_id, start_date)`, `INSERT … ON CONFLICT DO NOTHING`	—

ИНВАРИАНТ	ГДЕ ОБЕСПЕЧИВАЕТСЯ	ОТВЕТ ПРИ НАРУШЕНИИ
Одна кампания и одно утверждение графика на год; утверждение — до начала года и не пустое	Первичный ключ `year`; сервис планирования	`400 validation/invalid-field`
Автоматическое напоминание о планировании — одно	`UPDATE … SET auto_reminded = true WHERE NOT auto_reminded` в транзакции рассылки	—
Пароль не хранится и не возвращается	Только scrypt-хеш; DTO ответа без хеша	—
Версия данных растёт при любом изменении графика	Отложенные триггеры `AFTER INSERT/UPDATE/DELETE … DEFERRABLE INITIALLY DEFERRED FOR EACH ROW` (и `AFTER TRUNCATE` на оператор) на `vacation_requests`, `employees`, `departments`, `projects`, `work_patterns`, `work_overrides`, `org_settings` вызывают `bump_data_version()` при фиксации той же транзакции	—
Кэшированный ответ графика не опережает данные	Сервер читает версию до данных; кэш и ETag — по прочитанной версии	—

5 Индексы

ТАБЛИЦА	ИНДЕКС	ПОД КАКОЙ ЗАПРОС
`vacation_requests`	`(employee_id, status)`	Заявки сотрудника, проверка пересечений и лимита
`vacation_requests`	`(start_date, end_date)`	График и аналитика за период
`employees`	`(dept_id)`, `(manager_id)`, `(is_active)`	Фильтры графика, подчинённые, активные
`notifications`	`(employee_id, is_read, created_at)`	Колокольчик: непрочитанные по дате
`audit_events`	`(entity, entity_id, ts)`, `(actor_id)`	Журнал по объекту и по автору
`request_events`	`(request_id)`	История заявки
`email_outbox`	`(status, next_attempt_at)`	Выбор писем к отправке
`employees`	`lower(email)` уникальный частичный	Поиск и уникальность адреса
`login_attempts`	`(key, window_start)` первичный, `(window_start)`	Счётчик окна при входе; очистка старых окон
`login_events`	`(ts DESC)`, `(employee_id, ts DESC)`	Журнал входов по времени и по сотруднику; очистка по сроку
`refresh_tokens`	`(session_id)`	Сеансы сотрудника, проверка живости сеанса в guard
`approval_delegations`	`(manager_id, end_date)`, `(deputy_id, end_date)`	Действующие замещения руководителя и заместителя
`vacation_requests`	`(escalated_to)`	Очередь эскалированных заявок
`vacation_notices`	`(request_id, start_date)` уникальный, `(start_date)`, `(employee_id)`	Повтор задачи, журнал за период, свои извещения
`employees`	`(personnel_number)` уникальный	Табельный номер в карточке, импорте и поиске

6 Служебные таблицы масштабирования

Таблицы v0.5.0 обслуживают работу нескольких процессов и экземпляров сервера (ADR-012) и бизнес-данных не содержат.

Версия данных графика (`data_version`). Шестнадцать строк `(shard, version)`. Функция `bump_data_version()` увеличивает строку `shard = pg_backend_pid() % 16`: параллельные транзакции из разных соединений почти всегда обновляют разные строки и не ждут друг друга (с одной строкой p95 записи под нагрузкой вырос с 252 до 487 мс). Версия — `sum(version)`; она только растёт, поэтому годится для ETag и ключа кэша.

Триггеры отложенные: счётчик увеличивается в фазе фиксации и один раз за транзакцию (флаг `vp.data_version_bumped` с номером транзакции), сколько бы строк она ни изменила; оператор, не изменивший строк, версию не меняет. Поэтому строка счётчика заблокирована только на время фиксации. В первой редакции (триггеры на оператор) строка блокировалась сразу и до конца транзакции, и при совпадении строк счётчика две подачи заявок взаимно блокировались (`40P01`, найдено 15-минутным нагрузочным прогоном; миграция `data_version_on_commit`).

Окна лимита входа (`login_attempts`). Строка `(key, window_start, hits)` на ключ «адрес + логин» или «адрес» и минуту по часам БД. Вход увеличивает оба счётчика одним запросом (`INSERT … ON CONFLICT DO UPDATE`) и оценивает частоту скользящим окном: `hits + прошлая минута × доля, ещё не вышедшая из окна`. Таблица нежурналируемая (`UNLOGGED`): не пишется в WAL и не реплицируется, после аварийного перезапуска PostgreSQL очищается — для минутных счётчиков это допустимо. Окна старше 5 минут удаляются не чаще раза в 5 минут.

7 Производственный календарь

Таблица `calendars` хранит по году два разных понятия:

ПОЛЕ	ЧТО ЭТО	ГДЕ ВЛИЯЕТ
`holidays`	Нерабочие праздничные дни ст. 112 ТК РФ	Не входят в дни отпуска (ст. 120); подсвечиваются на графике
`transfers`	Переносы выходных `{ from, to }`: день `to` становится выходным; `from`, если это выходной и не праздник, — рабочим	Рабочие дни в аналитике, фон графиков; в дни отпуска перенесённый выходной входит

Праздник, выпавший на выходной, переносится на следующий рабочий день (ст. 112 ч. 2) — это перенос, а не праздник. Например, в 2026 году 9 мая (суббота) — праздник, а 11 мая (понедельник) — перенесённый выходной: отпуск 4–17 мая расходует 13 дней — исключается только 9 мая.

Календарь ведёт кадровая служба в «Справочниках»: отмечает праздники в сетке года и переносы или загружает файл открытых данных xmlcalendar.ru. Если календарь года не загружен, дни считаются календарными без исключений.

Исправление v0.4.0. Демо-календарь версий 0.1–0.3 смешивал праздники с выходными и перенесёнными днями (13–14 июня, 10 мая, 9 января и другие не входили в дни отпуска). Миграция `20261029091000_calendar_holidays_by_law` заменяет календари 2026 и 2027 годов на законные — только если они совпадают с демо-набором, изменённые вручную не трогает.

8 Миграция данных прототипа

`server/prisma/seed.ts` переносит снимок прототипа (`server/prisma/seed-data/seed.json`):

ДАННЫЕ	РЕЗУЛЬТАТ
131 сотрудник	Идентификаторы и логины сохранены; пароли — scrypt-хеши демо-значений (при эксплуатационной миграции — сброс на одноразовые)
14 отделов	Созданы из строковых названий; руководители — из словаря `deptManagers`
268 заявок	Перенесены со статусами, переносами и запросами удаления
Недельные шаблоны	Перенесены в `work_patterns`

Seed идемпотентен: повторный запуск не дублирует заявки (совпадение сотрудника, дат и типа) и ничего не удаляет, поэтому безопасен для рабочей базы. Для демонстрационного стенда seed добавляет несколько заявок на согласовании относительно текущей даты; переменная `SEED_DEMO=0` отключает их.

9 Изменение схемы

1. Изменение `schema.prisma` → `npx prisma migrate dev --name <суть>`; SQL миграции проходит ревью человека (CODEOWNERS).
2. Деструктивные изменения — в два выпуска: сначала новое поле и перенос данных, затем удаление старого.
3. Миграции применяются автоматически при старте контейнера `server` (`prisma migrate deploy`); откат — восстановление резервной копии (руководство по развёртыванию).
4. CI применяет миграции к чистой PostgreSQL перед тестами: рассинхронизация схемы и миграций ломает сборку. Отдельный шаг (`scripts/upgrade-check.sh`) применяет миграции `main`, наполняет базу данными прошлого выпуска (`scripts/upgrade-fixture.sql`), применяет новые миграции ветки и сверяет схему: так ловятся миграции, падающие только на данных.
5. Изменение данных таблицы с отложенным триггером версии (`UPDATE vacation_requests …`) в миграции ставится последним: после него `ALTER TABLE` и `CREATE INDEX` той же таблицы в той же транзакции падают с «pending trigger events».
6. После изменения схемы обновляются ER-схемы (`docs/diagrams/er-*.mmd`) и словарь данных (`tools/docs/gen_tables.py`).